

# كيف بُني النظام وكيف استخدم

## PMS — System Narrative

دليل نظري شامل — الإجابة على سؤال: ماذا صنعنا، لماذا، وكيف يعمل النظام ككل؟

منصة إدارة مشاريع التخرج · Multi-Tenant · React + Laravel + MySQL + Gemini

## فهرس المحتويات

1. المقدمة — ما هو PMS؟
2. المشكلة والحل — لماذا بُني هذا النظام؟
3. الفكرة العامة — كيف صار النظام؟
4. من يستخدم النظام؟ — الأدوار والعلاقات
5. كيف يعمل النظام نظرياً — النموذج العام
6. رحلة المستخدم — من التسجيل إلى إنجاز المشروع
7. كيف بُني النظام — الطبقات والقرارات
8. كيف استُخدمت المزايا — شرح نظري لكل مجال
9. التعاون داخل المشروع — قلب النظام
10. الذكاء الاصطناعي في سياق النظام
11. الأمان والعزل — كيف نحمي البيانات
12. ما لم نبنيه ولماذا — حدود النطاق
13. أسئلة وأجوبة للمناقشة
14. الخلاصة والوثائق المرافقة

## 1. المقدمة — ما هو PMS؟

**PMS** (Project Management System) منصة ويب لإدارة مشاريع التخرج والمشاريع الجامعية. ليست قائمة مهام بسيطة، بل مساحة عمل تعاونية تجمع الطالب ومشرفه وفريقه ومدير الجامعة في مكان واحد — لكل جامعة بيئة معزولة عن الجامعات الأخرى.

النظام يُجيب نظرياً على سؤال: «كيف يُدار مشروع التخرج من الفكرة حتى التسليم، ضمن إطار جامعي منظم وآمن؟»

**بجملته واحدة:** PMS منصة Multi-Tenant تربط أصحاب المصلحة في مشاريع التخرج، وتوفّر أدوات التخطيط والتنفيذ والتواصل والمتابعة — مع مساعدة ذكاء اصطناعي في مرحلة الفكرة والتخطيط.

## 2. المشكلة والحل — لماذا بُني هذا النظام؟

### 2.1 المشكلة (قبل النظام)

- تشتت التواصل بين الطالب والمشرف (واتساب، إيميل، ملفات متفرقة).
- صعوبة متابعة تقدم المشروع ومهام الفريق بشكل موحد.
- لا يوجد سجل مركزي للنشاطات والإصدارات والملفات.
- كل جامعة تحتاج بيئة خاصة دون خلط بيانات مع جامعات أخرى.
- الطالب يحتاج مساعدة في اختيار فكرة المشروع وتقسيمه لمهام قابلة للتنفيذ.

## 2.2 الحل (ما قدّمه PMS)

| الحاجة        | كيف أجاب النظام                           |
|---------------|-------------------------------------------|
| تنظيم العمل   | مشروع مركزي + مهام + نسبة إنجاز           |
| بناء الفريق   | دعوات مشرف وطلاب مع قبول/رفض              |
| التواصل       | تعليقات على مستوى المشروع + إشعارات       |
| التسليمات     | إصدارات ملفات + ربط GitHub                |
| الشفافية      | سجل نشاط (Activity Log) لكل حدث مهم       |
| حوكمة الجامعة | موافقة حسابات + إدارة مستخدمين لكل tenant |
| تعدد الجامعات | Multi-Tenancy — عزل بيانات كامل           |
| مساعدة الطالب | AI: اقتراح أفكار + توليد مهام             |

## 3. الفكرة العامة — كيف صار النظام؟

تحوّلت الفكرة إلى نظام عبر مراحل منطقية:

### المرحلة 1 — التأسيس (الهيكل الأساسي)

بدأنا ببناء الأساس: مستخدمون بأدوار مختلفة، جامعات ك tenants، مصادقة آمنة، وإنشاء مشاريع. الهدف: أي مستخدم يسجل ويوافق عليه ويستطيع العمل ضمن جامعتة فقط.

### المرحلة 2 — التعاون (الفريق والمشروع)

أضفنا آليات التعاون: دعوة مشرف وطلاب، مهام بحالات (pending / in\_progress / completed)، تعليقات الفريق، ونسبة تقدم تُحسب من المهام. المشروع أصبح «مركز العمل» وليس مجرد سجل.

### المرحلة 3 — التسليم والمتابعة

أضفنا الإصدارات والملفات وتكامل GitHub لمشاريع البرمجة، وسجل النشاط لتوثيق من فعل ماذا ومتى. المشرف يتابع دون الحاجة لطلب تقارير يدوية.

### المرحلة 4 — الحوكمة والمنصة

أضفنا لوحات تحكم حسب الدور، إدارة المستخدمين، موافقة الحسابات، و Super Admin لإدارة عدة جامعات على منصة واحدة.

## المرحلة 5 — الذكاء الاصطناعي

دمجنا **Google Gemini** كمساعد خارجي: اقتراح أفكار مشاريع من اهتمامات الطالب، وتقسيم وصف المشروع إلى مهام. AI يُسرّع البداية — القرار النهائي للطالب.

Problem → Requirements → Architecture (3-tier SPA + API + DB)  
→ Core (Auth + Multi-Tenant + Projects)  
→ Collaboration (Invites + Tasks + Comments)  
→ Delivery (Versions + GitHub + Activity)  
→ Governance (Dashboards + Admin + Approvals)  
→ AI Assistant (Ideation + Task Generation)  
→ i18n (Arabic / English)

## 4. من يستخدم النظام؟ — الأدوار والعلاقات

| الدور              | من هو؟           | دوره في النظام                                                            |
|--------------------|------------------|---------------------------------------------------------------------------|
| <b>Student</b>     | طالب جامعي       | ينشئ المشروع، يدعو الفريق والمشرف، يدير المهام، يستخدم AI، يرفع الإصدارات |
| <b>Supervisor</b>  | عضو هيئة تدريس   | يقبل دعوات الإشراف، يتابع المشاريع، يشارك في النقاش، يراجع التقدم         |
| <b>Admin</b>       | مدير جامعة واحدة | يوافق على الحسابات الجديدة، يدير مستخدمي جامعتهم، يتابع المشاريع          |
| <b>Super Admin</b> | مدير المنصة      | يدير الجامعات والمستخدمين والمشاريع على مستوى المنصة ككل                  |

العلاقة بينهم ليست هرمية صارمة — الطالب «مالك» مشروعه، المشرف «شريك إشراف»، والمدير «حارس البوابة» لحسابات الجامعة. Super Admin فوق كل ال tenants.

## 5. كيف يعمل النظام نظرياً — النموذج العام

### 5.1 النمط المعماري (بكلمات بسيطة)

النظام يتبع نموذج **Client-Server** بثلاث طبقات:

1. **طبقة العرض (Presentation):** تطبيق React في المتصفح — ما يراه المستخدم ويتفاعل معه.
2. **طبقة الأعمال (Business):** Laravel API — يتحقق من الصلاحيات وينقذ القواعد.
3. **طبقة البيانات (Data):** MySQL — يخزن المستخدمين والمشاريع والمهام وغيرها.

كل طلب من المستخدم يمر: واجهة → API → منطق أعمال → قاعدة بيانات → رد JSON → تحديث الواجهة.

## 5.2 مبدأ Multi-Tenancy

عدة جامعات تستخدم نفس التطبيق ونفس قاعدة البيانات، لكن بيانات كل جامعة معزولة منطقياً عبر `university_id`. طالب جامعة A لا يرى ولا يصل لمشاريع جامعة B — تلقائياً في كل استعلام.

## 5.3 مبدأ الصلاحيات المتعددة الطبقات

لا يكفي أن يكون المستخدم «مسجل دخول» — النظام يتحقق بالتسلسل:

1. هل الـ token صالح؟ (Sanctum)
2. هل له جامعة؟ (EnsureUserHasUniversity)
3. هل حسابه active وليس pending/rejected؟
4. هل دوره يسمح بهذه العملية؟ (role middleware)
5. هل له حق الوصول لهذا المشروع تحديداً؟ (ProjectAccess)

## 5.4 المشروع كوحدة مركزية

كل التعاون يدور حول صفحة تفاصيل المشروع — تبويبات: نظرة عامة، دعوات، مهام، تعليقات، إصدارات، سجل نشاط. هذا التصميم يجعل النظام مفهوماً: «ادخل مشروعك واعمل من هناك».

## 6. رحلة المستخدم — من التسجيل إلى إنجاز المشروع

### 6.1 رحلة الطالب (السيناريو الكامل)

1. التسجيل: يختار جامعته ودوره (طالب) → حساب pending.
2. الانتظار: ينتظر موافقة مدير الجامعة — يرى شاشة انتظار فقط.
3. التفعيل: بعد الموافقة → لوحة تحكم + قائمة مشاريع.
4. الفكرة (اختياري): صفحة AI Ideation → يكتب اهتماماته → 3 اقتراحات → يحفظ أو يعتمد فكرة.
5. إنشاء المشروع: عنوان + وصف → مشروع جديد مرتبط بجامعته.
6. بناء الفريق: يدعو مشرفاً وطلاباً → ينتظر قبولهم.
7. التخطيط: يولّد مهام بالذكاء الاصطناعي أو يضيفها يدوياً.
8. التنفيذ: يغيّر حالة المهام → التقدم يتحدث → الإشعارات تُرسل للفريق.
9. التواصل: تعليقات على المشروع بين أعضاء الفريق.
10. التسليم: رفع إصدارات/ملفات → دفع لـ GitHub إن وُجد.
11. المتابعة: المشرف يرى Activity Log والتقدم دون اجتماعات إضافية.

## 6.2 رحلة المشرف

1. يسجل ويختار جامعة/جامعات → pending لكل جامعة على حدة.
2. بعد الموافقة → يرى دعوات إشراف واردة.
3. يقبل دعوة → يصبح مشرفاً على المشروع.
4. يتابع المهام والتعليقات والإصدارات من تبويبات المشروع.
5. يمكنه مغادرة الإشراف إن انسحب.

## 6.3 رحلة مدير الجامعة

1. يدير مستخدمي جامعتهم: عرض، إنشاء، تعديل، حذف.
2. يراجع الحسابات المعلقة (pending) ويوافق أو يرفض.
3. يتعامل مع طلبات إعادة تعيين كلمة المرور.
4. يرى إحصائيات الجامعة في لوحة التحكم.

**الفكرة النظرية:** كل دور له «بوابة دخول» مختلفة لكنهم يلتقون داخل **صفحة المشروع** — نفس البيانات، صلاحيات مختلفة.

## 7. كيف بُني النظام — الطبقات والقرارات

### 7.1 Frontend — كيف صُمِّم؟

اخترنا **React SPA** لأن الواجهة تفاعلية (تبويبات، فلاتر، إشعارات) وتحتاج تحديثاً سريعاً دون إعادة تحميل الصفحة. استخدمنا:

- **Context API** لحالة المصادقة واللغة — بسيط وكافي دون Redux.
- **MUI** لمكونات جاهزة ومتسقة.
- **React Router** للتنقل بين الصفحات.
- **i18n** ملفات عربي/إنجليزي مع دعم RTL.

كل صفحة = مسؤولية واضحة: Login، Projects، ProjectDetails، Users، Ideation، ...

### 7.2 Backend — كيف صُمِّم؟

اخترنا **Laravel Monolith** مع **Service Layer: Controller** رفيع (validation + response) و Service يحمل المنطق (AuthService، ProjectService، AITaskService، ...).

- لماذا **Monolith**؟ حجم المشروع الأكاديمي لا يبرر تعقيد microservices.
- لماذا **Services**؟ فصل المنطق عن HTTP يسهل الاختبار والصيانة.

- لماذا Sanctum؟ SPA منفصلة عن API تحتاج token stateless.
- لماذا Eloquent + TenantScope؟ عزل tenant تلقائي دون تكرار فلاتر في كل query.

### 7.3 قاعدة البيانات — كيف صُمّمت؟

نموذج علائقي في MySQL (~22 جدول). الكيانات المركزية: `users` ، `universities` ، `projects` ، `tasks` ، `project_members` ، `invitations` ، `comments` ، `project_versions` ، `project_activities` ، `notifications`. العلاقات تعكس الواقع: مشروع ينتمي لجامعة، مهام تنتمي لمشروع، أعضاء يرتبطون بدعوات.

### 7.4 قرارات تقنية رئيسية

| القرار                 | البديل المرفوض      | المبرر                          |
|------------------------|---------------------|---------------------------------|
| Shared DB Multi-Tenant | DB منفصلة لكل جامعة | أبسط في النشر والصيانة          |
| REST API               | GraphQL             | أبسط للفريق والمشروع الأكاديمي  |
| ProjectAccess helper   | Laravel Policies    | مركزية أسرع للتنفيذ الحالي      |
| Sync AI requests       | Queue + Jobs        | كافي للنطاق الحالي؛ أبسط        |
| Database notifications | WebSockets          | لا حاجة لبنية real-time معقدة   |
| Gemini HTTP API        | ML محلي             | لا بنية ML؛ جودة عالية بجهد أقل |

## 8. كيف استُخدمت المزايا — شرح نظري لكل مجال

هذا القسم يجيب: «ماذا يفعل المستخدم ولماذا وُجدت هذه الميزة؟» — دون تفاصيل كود.

### 8.1 المصادقة والحسابات

الاستخدام النظري: البوابة الآمنة للنظام. لا أحد يعمل قبل التحقق من هويته وحالة حسابه. التسجيل الذاتي يفتح الباب — الموافقة الإدارية تضمن أن فقط أعضاء الجامعة المعتمدين يدخلون.

### Multi-Tenancy 8.2

الاستخدام النظري: يسمح لمنصة واحدة بخدمة عدة جامعات كأن لكل منها نظامها الخاص — دون تكلفة نشر منفصل. أساس الثقة: بياناتي لا تختلط ببيانات غيري.



### 8.3 إدارة المشاريع

**الاستخدام النظري:** المشروع هو «حاوية العمل» — عنوان، وصف، تقدم، أعضاء، مشرف. الطالب يملك مشروعه؛ النظام يحسب التقدم من المهام المكتملة تلقائياً.

### 8.4 الدعوات

**الاستخدام النظري:** بناء الفريق لا يكون بإضافة عشوائية — المدعو يقبل أو يرفض. هذا يحترم المشرف والطالب المدعو ويمنع انضماماً غير مرغوب.

### 8.5 المهام

**الاستخدام النظري:** تحويل المشروع الكبير لخطوات صغيرة قابلة للتتبع. الحالات الثلاث (معلقة / قيد التنفيذ / مكتملة) تعطي صورة واضحة للتقدم للطالب والمشرف.

### 8.6 التعليقات

**الاستخدام النظري:** نقاش الفريق داخل المشروع — بديل منظم للمحادثات الخارجية. كل التعليقات مرتبطة بالمشروع ومرئية لمن لهم صلاحية الوصول.

### 8.7 الإصدارات و GitHub

**الاستخدام النظري:** توثيق التسليمات (ملفات، نسخ) وربطها بمستودع GitHub لمشاريع البرمجة. المشرف يرى ماذا سُلّم ومتى دون طلب الملفات عبر قنوات أخرى.

### 8.8 سجل النشاط (Activity Log)

**الاستخدام النظري:** الذاكرة الزمنية للمشروع — من أنشأ مهمة، من غيّر حالة، من رفع إصداراً. ليس بديلاً لقائمة الإصدارات — بل سجل أوسع لكل الأحداث.

### 8.9 الإشعارات

**الاستخدام النظري:** إبقاء الفريق على اطلاع دون الحاجة لفتح المشروع باستمرار. حدث مهم → إشعار → رابط للمشروع/المهمة.

### 8.10 لوحات التحكم

**الاستخدام النظري:** نقطة بداية مخصصة لكل دور — ملخص سريع قبل الغوص في التفاصيل. Badges للدعوات المعلقة والإشعارات غير المقروءة.

8.11 إدارة المستخدمين

الاستخدام النظري: أداة الحوكمة — من يدخل النظام؟ من يُعلّق؟ من يحتاج كلمة مرور جديدة؟ مدير الجامعة يدير جامعته؛ Super Admin يدير المنصة.

8.12 التوطين (i18n)

الاستخدام النظري: النظام يخدم مستخدمين عرباً وإنجليزاً — الواجهة والاتجاه (RTL/LTR) يتغيران. حتى تعليمات AI تتكيف مع لغة إدخال المستخدم.

9. التعاون داخل المشروع — قلب النظام

صفحة تفاصيل المشروع هي المحور الذي يجمع كل المزايا:

| التبويب  | الغرض النظري                              | من يستخدمه؟    |
|----------|-------------------------------------------|----------------|
| Overview | نظرة شاملة: معلومات، تقدم، أعضاء، commits | الكل           |
| Invites  | توسيع الفريق                              | مالك المشروع   |
| Tasks    | التخطيط والتنفيذ + AI                     | الفريق         |
| Comments | النقاش والتواصل                           | الفريق         |
| Releases | التسليمات والملفات                        | الفريق         |
| Activity | الشفافية والتدقيق                         | المشرف والفريق |

التصميم النظري: بدلاً من صفحات متفرقة لكل ميزة، جمعنا التعاون في مكان واحد — يقلل التشتت ويعكس واقع عمل مشروع التخرج: كل شيء يدور حول المشروع نفسه.

10. الذكاء الاصطناعي في سياق النظام

AI ليس «النظام كله» — بل طبقة مساعدة في مرحلتين:

- 1. قبل المشروع: Ideation — الطالب لا يعرف ماذا يبني → AI يقترح 3 أفكار.
  - 2. بعد إنشاء المشروع: Task Generation — وصف المشروع موجود → AI يقسّمه لمهام.
- المبدأ: AI — **Human-in-the-loop** يقترح، الإنسان يقرر. لا يُنشأ مشروع أو تُحذف مهام دون موافقة صريحة.
- التفاصيل التقنية في [PMS-AI-GUIDE.pdf](#).

## 11. الأمان والعزل — كيف نحمي البيانات

| التهديد النظري            | الحماية                                   |
|---------------------------|-------------------------------------------|
| وصول غير مصرح             | Sanctum token + middleware سلسلة          |
| تسريب بيانات جامعة أخرى   | TenantScope + university_id               |
| وصول لمشروع لست عضواً فيه | ProjectAccess                             |
| إساءة استخدام AI          | Rate limits + student only + sanitization |
| حسابات وهمية              | موافقة admin قبل active                   |

## 12. ما لم نبهه ولماذا — حدود النطاق

- لا دردشة فورية (WebSockets): الإشعارات عند الطلب كافية للنطاق الأكاديمي.
- لا تطبيق موبايل SPA responsive: native تكفي.
- لا نظام درجات/تقييم: خارج نطاق إدارة المشروع.
- لا microservices: تعقيد غير مبرر.
- لا تعليقات مهام في UI: التعليقات على مستوى المشروع تغطي الحاجة الحالية.

هذه الحدود قرارات واعية — النظام يركز على **collaboration** لمشاريع التخرج وليس كل احتياج جامعي.

## 13. أسئلة وأجوبة للمناقشة

### س: ما هو PMS باختصار؟

ج: منصة ويب Multi-Tenant لإدارة مشاريع التخرج — تعاون، مهام، تسليمات، إشعارات، ومساعد AI.

### س: كيف صار النظام؟ ما المنهجية؟

ج: بناء تدريجي: أساس (auth + tenant) → تعاون (مشاريع + مهام) → تسليم (إصدارات + GitHub) → حوكمة (admin) → AI. كل مرحلة تبني على السابقة.

### س: كيف استخدمتم Multi-Tenancy؟

ج: قاعدة بيانات مشتركة + university\_id على كل سجل + TenantScope يفلتر تلقائياً. Super Admin يتجاوز العزل.

### س: لماذا React + Laravel وليس WordPress أو حل جاهز؟

ج: الحاجة مخصصة: أدوار متعددة، دعوات، AI، GitHub، tenant isolation — لا يوفرها قالب جاهز بمرونة كافية.

### س: كيف يتعامل النظام مع المشرف في عدة جامعات؟

ج: جدول pivot supervisor\_universities — موافقة منفصلة لكل جامعة. الحالة النهائية للحساب تُحسب من مجموع العضويات.

### س: ما الفرق بين Activity Log وقائمة الإصدارات؟

ج: الإصدارات = ملفات مرفوعة فقط. Activity Log = كل الأحداث (مهام، دعوات، توليد AI، ...).

### س: ما دور الذكاء الاصطناعي؟ هل يستبدل الطالب؟

ج: لا. يقترح أفكاراً ومهاماً — الطالب يختار ويعتمد ويعدل. مساعد في البداية وليس بديلاً.

### س: كيف تُحسب نسبة تقدم المشروع؟

ج: نسبة المهام ذات الحالة completed من إجمالي المهام — تحديث تلقائي عند تغيير أي مهمة.

### س: ما أبرز قيود النظام الحالي؟

ج: AI متزامن (sync) · لا real-time · monolith · helper وليس Policies — مقبول للنطاق الأكاديمي.

س: كيف يضمن النظام أن الطالب يرى مشاريع جامعته فقط؟

ج: عند التسجيل يُربط بجامعة · TenantScope يضيف WHERE university\_id تلقائياً · لا حاجة لفلتر يدوي في كل صفحة.

## 14. الخلاصة والوثائق المرافقة

### 14.1 الخلاصة

**PMS** نظام بُني لحل مشكلة حقيقية: إدارة مشاريع التخرج بشكل منظم ومتعدد الجامعات. بُني تدريجياً من الأساس إلى التعاون إلى التسليم إلى الحوكمة إلى AI. يُستخدم نظرياً ك **مساحة عمل جماعية** حول المشروع — كل ميزة تخدم هدفاً: تنظيم، تواصل، شفافية، أو مساعدة.

#### للمناقشة — الجملة الذهبية:

«بنينا منصة collaboration لمشاريع التخرج على بنية Multi-Tenant آمنة، بفصل واضح بين العرض والأعمال والبيانات، ودمجنا الذكاء الاصطناعي كمساعد في مرحلة الفكرة والتخطيط — مع إبقاء القرار النهائي للمستخدم.»

### 14.2 خريطة الوثائق المرافقة

| الوثيقة                                 | ماذا تجيب؟                                   |
|-----------------------------------------|----------------------------------------------|
| <b>PMS-SYSTEM-NARRATIVE</b> (هذا الملف) | نظري شامل — كيف بُني وكيف استُخدم النظام ككل |
| PMS-PROJECT-SCOPE                       | ماذا داخل النطاق وماذا خارجه                 |
| PMS-ARCHITECTURE-CHAPTER                | فصل المعمارية التقني للرسالة                 |
| PMS-FEATURES-USAGE                      | كيف تُفُذت كل ميزة (UI → API → Service)      |
| PMS-AI-GUIDE                            | تفاصيل الذكاء الاصطناعي                      |
| PMS-ACTIVITY-DIAGRAM                    | مخططات نشاط لكل عملية                        |
| PMS-ERD-FULL                            | مخطط قاعدة البيانات                          |